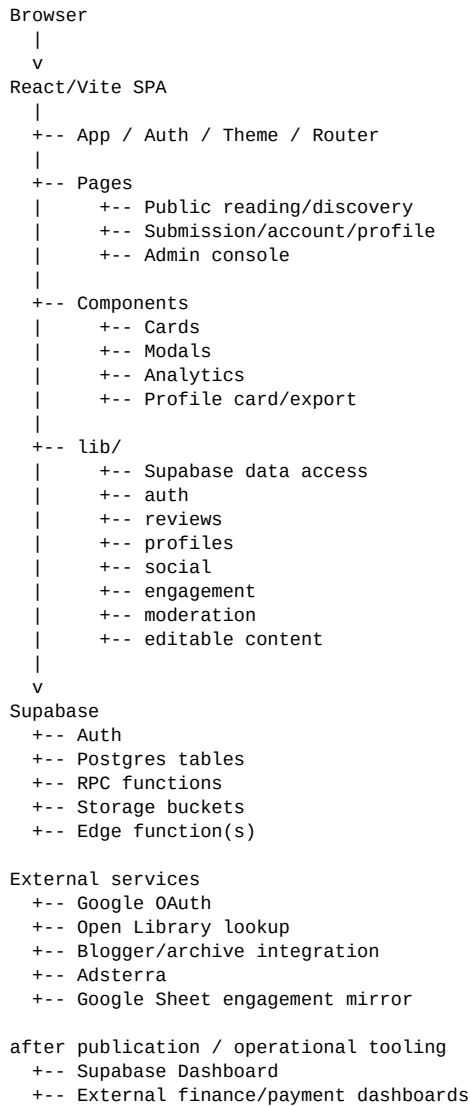


Novelty Library WebApp — Architecture

Source: NL_WA_1_75 from NL_WA_1_35.zip

1. Architecture overview

Novelty Library is a client-rendered React 18 single-page application built with Vite and TypeScript.



2. Technology stack

Runtime

- React 18.3.
- React DOM 18.3.
- TypeScript 5.5.
- Vite 5.4.
- Tailwind CSS 3.4.
- Framer Motion.

- Lucide React.
- DOMPurify.
- html-to-image.
- vite-plugin-pwa.

Backend

- Supabase JS 2.x.
- Supabase Auth.
- Supabase Postgres.
- Supabase Storage.
- Supabase RPC/database functions.
- Supabase Edge Function for engagement-to-sheet synchronization.

3. Application bootstrap

`src/main.tsx` mounts the React application.

`src/App.tsx` composes:

- ThemeProvider.
- AuthProvider.
- AppContent.
- Global header/footer.
- Cookie banner.
- Install prompt.
- Splash intro.
- Lazy onboarding modal.

An onboarding error boundary prevents a broken onboarding module from breaking the entire application and records a local dismissal flag.

4. Routing architecture

The app uses a custom hash router rather than React Router.

`src/lib/router.ts` parses `window.location.hash` into a typed `Route` union. Navigation writes directly to `window.location.hash`.

Major route groups:

Public

- /
- /reviews
- /reviews + genre filtering
- /review/:slug
- /blog-review/:id

- /analytics
- /submit
- /review-guidelines
- /about
- /privacy
- /terms
- /cookies
- /auth
- /profile
- /profile/:username
- /account
- /notifications

Admin

- /admin
- /admin/book-reviews/:tab
- /admin/reviews
- /admin/master-list
- /admin/posters
- /admin/admins
- /admin/users
- /admin/finances
- /admin/pages

The route parser also supports action fragments such as the submit-page reservation action.

5. Authentication architecture

`src/lib/auth.tsx` is the application-wide auth context.

Supported flows:

- Email/password sign-in.
- Email/password registration.
- Email confirmation resend.
- Google OAuth.
- Password reset.
- Password update after recovery.
- Sign out.
- Profile refresh.

Important invariant:

There must be exactly one Supabase client/GoTrueClient instance in the browser.

`src/lib/communitySupabase.ts` intentionally re-exports the primary client instead of creating a second client.

Admin status is resolved with the backend `is_admin()` RPC.

6. Data-access architecture

The `src/lib` layer is the application's data-access/service layer. Pages should consume these functions rather than duplicating raw Supabase queries unnecessarily.

Key modules:

Module	Responsibility
<code>supabase.ts</code>	Primary client
<code>auth.tsx</code>	Auth state and account operations
<code>reviews.ts</code>	Review lifecycle, master list, publication, profiles, reservations
<code>engagement.ts</code>	Community likes/ratings/reviews and NL Rating
<code>profileQuestions.ts</code>	Configurable profile question schema/configuration
<code>profileCoreFields.ts</code>	Built-in profile field overrides/order
<code>profileLayout.ts</code>	Profile section ordering/activation
<code>social.ts</code>	Follows, stories and social interactions
<code>notifications.ts</code>	Notification retrieval/read state
<code>moderation.ts</code>	Reports, grievances and blacklist checks
<code>adminConfig.ts</code>	Editable pages and finance modules
<code>aboutSections.ts</code>	About-page structured content
<code>reviewGuidelines.ts</code>	Review-guideline content
<code>homeAnalytics.ts</code>	Aggregate homepage/analytics RPC
<code>blog.ts</code>	Blogger/archive retrieval and normalization
<code>imageUpload.ts</code>	Profile question image handling
<code>siteSettings.ts</code>	Site-wide settings such as support email
<code>cookieConsent.ts</code>	Cookie consent persistence
<code>sanitize.ts</code>	User text and URL safety

7. Supabase data model inferred from the code

Tables

- `profiles`
- `reviews`
- `master_list`
- `community_reviews`
- `review_drafts`
- `book_reservations`
- `profile_questions`

- profile_question_sections
- editable_pages
- about_sections
- review_guideline_sections
- finance_modules
- community_review_feedback
- community_review_likes
- community_review_stats
- community_review_comments
- community_review_comment_likes
- community_review_feedback_replies
- stories
- story_likes
- story_reposts
- notifications
- user_grievances

Storage buckets referenced

- avatars
- covers
- stories
- profile-question-images

RPC/database functions referenced

- is_admin
- home_analytics
- check_book_availability
- my_accepted_reviews
- my_published_reviews
- public_profile_by_username
- public_accepted_reviews_by_username
- public_published_reviews_by_username
- admin_save_profile_question_section_v2
- admin_save_profile_question_v2
- admin_list_users
- admin_add_admin
- admin_list_admins
- admin_remove_admin
- admin_list_blacklisted_users
- is_profile_blacklisted
- submit_user_report

- submit_grievance
- get_follow_stats
- follow_username
- unfollow_username
- get_profile_connections
- get_profile_privacy
- repost_story
- get_site_support_email
- set_site_support_email

8. Review data flow



The code contains explicit publication functions that copy/upsert publication-oriented data into `master_list` and `community_reviews`.

9. Community engagement flow

`community_review_feedback` stores a user's rating/review/tags for a review post.

`community_review_likes` stores likes.

`community_review_stats` supplies aggregate counts.

NL Rating is computed as:

$$\text{NL Rating} = \text{ratingSum} / \text{ratingCount}$$

when at least one community rating exists.

The original reviewer's R/W Rating is deliberately excluded from this calculation.

After engagement mutation, `engagement - sheet` is invoked as a best-effort mirror. Failure of the mirror must not break the user-facing operation.

10. Profile architecture

Profiles combine fixed fields with dynamic question answers.

Fixed fields

Stored in `profiles` and represented by `CORE_FIELDS`.

Dynamic fields

Stored in `profiles.profile_answers` as a JSON-like object keyed by question key.

Admin configuration

`editable_pages` is used to store JSON configuration for:

- core-field overrides.
- core-field ordering.
- profile section layout.

This avoids introducing a table for every small presentation setting.

11. Profile-card export architecture

`ProfileCard.tsx` renders a fixed pixel canvas based on the selected aspect ratio. The page scales that canvas down for preview.

Export pipeline:

1. Set final export format.
2. Disable motion for export.
3. Wait for render/fonts.
4. Warm image/font loading.
5. Convert DOM canvas to PNG using `html-to-image`.
6. Trigger browser download.

A progressive fitting algorithm reduces scale and review count when the content exceeds the canvas.

12. Security boundaries

The client uses only the Supabase anonymous/public key from Vite environment variables.

Sensitive authorization must be enforced server-side/RLS/RPC-side. Client-side `isAdmin` is a UI gate, not a substitute for database authorization.

User text is sanitized with the application sanitization helpers.

External URLs are passed through safe URL validation before use.

13. Failure/degradation strategy

The current implementation deliberately treats some integrations as non-critical:

- Community engagement statistics can fail without preventing review pages from rendering.
- Google Sheet synchronization is fire-and-forget.
- Blog archive enrichment can fail after the primary review data has already rendered.
- About content falls back to built-in defaults if the database has no usable content.
- Profile layout/core configuration falls back to built-in defaults.

14. Deployment architecture

The project contains:

- `vercel.json`.
- Vite build scripts.
- PWA configuration.
- Public redirect configuration.

Expected deployment model is a static/client-rendered Vite build hosted on a platform such as Vercel, with Supabase providing the backend.

15. Architectural constraints

1. Do not introduce a second Supabase client.
2. Keep database authorization independent of UI visibility.
3. Keep public analytics aggregate-only.
4. Keep reviewer rating and community rating semantically separate.
5. Keep external blog/archive enrichment optional.
6. Keep admin configuration backwards-compatible with built-in defaults.
7. Preserve hash-route compatibility unless a migration is explicitly planned.